Joana Wang - *60225* & Renato Viola - *60665*

Implementing the MicroML language

Introduction

This report details the implementation of MicroML, an *OCaml*-like functional and imperative programming language, in *Java*.

To achieve this, there are several components at play, those being: a parser with an interpreter, typechecker, and compiler.

The following sections will cover instructions on how to execute programs in this language, its syntax, semantic deviations to what was presented in class, and lastly some relevant implementation details.

Executing MicroML programs

The project's repository includes three shell scripts to automate the execution process, each with its own purpose and usage.

IMPORTANT: If using the *Windows OS*, please use *Git Bash* instead of the *Windows Powershell*.

build.sh

This script generates the *Java* sources from the parser's *JavaCC* grammar and then compiles the project's source files, including those generated by *JavaCC*.

Note: There is likely no need to use this script explicitly, as it is already invoked in the other two.

Usage: *bash build.sh*

interpret.sh

This script compiles the project, starts the parser and then executes the typechecker followed by the interpreter, outputting the result in the console.

Note: Requires the path to a text file containing the MicroML program to evaluate.

Usage: *bash interpret.sh <path_to_file_input>*

compile.sh

This script compiles the project, starts the parser and then executes the typechecker followed by the compiler, generating the bytecode, decompiles it using *Jasmin*, and runs it, outputting the result in the console.

Note: Requires the path to a text file containing the MicroML program to evaluate.

Usage: *bash compile.sh <path_to_file_input>*

Concrete syntax accepted by the parser

Literals and Identifiers

```
int                                 // Integer literals (e.g., 123)
bool                                // Boolean literals (true, false)
id                                  // Identifiers (variable names)
```

Unit Expression

```
()                                  // Unit value
```

Arithmetic Expressions

```
E + E                               // Addition
E - E                               // Subtraction
E * E                               // Multiplication
E / E                               // Division
-E                                  // Unary negation
(E)                                 // Parenthesized expressions
```

Comparison Expressions

```
E == E                              // Equality
E != E                              // Inequality
E > E                               // Greater than
E >= E                              // Greater than or equal to
E < E                               // Less than
E <= E                              // Less than or equal to
```

Logical Expressions

```
E && E                              // Logical AND
E || E                              // Logical OR
~E                                  // Logical NOT
```

Let Expressions

```
let (id(: (T))? = E)+ in E end      // Let bindings, with optional type annotations
```

Memory Operations

```
new E                    // Allocation
E := E                   // Assignment
!E                       // Dereferencing
```

Conditional Expressions

```
if E then E else E end   // If-then-else
if E then E end          // If-then
```

Loop Expressions

```
while E do E end         // While loop
```

Print Statements

```
println E                // Print with newline
print E                  // Print without newline
```

Sequence Expressions

```
E; E                     // Sequence of expressions
```

Function Definitions

```
fun PL -> E end          // Function definition
E(EL?)                   // Function application
```

Expression List (EL)

```
E(, E)*                  // Comma-separated list of expressions
```

Parameter List (PL)

```
(id: (T))+               // One or more parameter declarations with types
```

Types (T)

```
unit                     // Unit type
int                      // Integer type
bool                     // Boolean type
ref T                    // Reference type
(T)* -> T                // Function type
```

Deviations in semantics from the original features

There are no significant deviations in terms of semantics from the features discussed in class.

Extras

Due to time constraints and other pending projects, we haven't implemented additional features for the *MicroML* language aside from what was mandatory.

Notable implementation details

We've employed the visitor pattern for traversing the AST during interpretation, type checking, and compilation, promoting code reuse and separation of concerns.

The type checker traverses the AST to not only ensure type safety and correct type declaration but also to add type information to certain nodes. This information is critical to the compiler later on.

Conclusion

Though we acknowledge that our implementation has some room for improvement, namely in the lack of additional functionalities, we are pleased by the results of our effort and believe that our solution is a competent attempt in meeting the handout's requirements.